

SMARTBRIDGE

Let's Bridge the Gap | SKILL WALLET - A SmartBridge Product

Date	29-09-2026
Team ID	SWTID-2026-7949
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value

Brainstorming & Idea Prioritization Template

Step 1: Brainstorm and Idea Listing

Each team member lists out as many ideas as possible without judging them at this stage.

S.No	Team Member	Idea / Suggestion	Category	Group No.
1	Team Member 1	Implement Script-Controlled Read ACL on u_institution_details with Branch == 'EEE' condition and gs.hasRole('bb1') check.	Security & Access Control	G-1
2	Team Member 2	Create dedicated granular custom roles (bb1 for Read, bb2 for Create, bb3 for Write, bb4 for Delete) mapped to departmental user profiles.	Role-Based Access Management	G-2
3	Team Member 3	Configure Client Scripts and UI Policies to hide non-EEE records dynamically on the form client-side.	User Interface / Client Scripting	G-3
4	Team Member 1	Elevate security_admin role and establish server-side Create, Write, and Delete ACLs to govern full CRUD lifecycle on custom tables.	System Security & Governance	G-1
5	Team Member 2	Utilize Business Rules (Before Query) to filter out records where Branch is not EEE at database query level.	Query Optimization & Security	G-4
6	Team Member 3	Implement automated impersonation testing for admin, EEE user, and unauthorized roles to validate zero record leakage across lists and forms.	Testing & Verification	G-5

Step 2: Idea Prioritization

Rate each grouped idea on feasibility and importance, then select the final idea(s) to move forward with.

Group No.	Final Idea	Feasibility (High/Medium/Low)	Importance (High/Medium/Low)	Priority	Selected (Yes/No)
1	Implement Script-Controlled ACLs on custom table (u_institution_details) combining data conditions (Branch is EEE) with script logic (gs.hasRole('admin') / gs.hasRole('bb1')) across	High	High	P1	Yes

	Read, Create, Write, Delete operations.				
2	Design and assign discrete custom security roles (bb1: Read, bb2: Create, bb3: Write, bb4: Delete) mapped directly to branch users to enforce least privilege.	High	High	P1	Yes
3	Rely on Client Scripts and UI Policies to restrict branch visibility on forms and list views.	High	Low	P4	No (Client-side checks are insecure and easily bypassed via API/reports)
4	Implement 'before query' Business Rules as a supplementary filter for record listings.	Medium	Medium	P3	No (ACLs provide native, auditable, ServiceNow security best practice for row-level security)
5	Perform structured User Impersonation testing across EEE User, Non-role User, and System Administrator to verify complete security isolation.	High	High	P2	Yes